

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

Факультет информационных технологий
Кафедра информатики и информационных технологий

Направление подготовки 09.03.02 «Информационные системы и технологии»

ОТЧЕТ
по проектной практике

Тема: реализация стека TCP/IP поверх Ethernet

Выполнил: Утюшев В.Э.,
(ФИО)

Группа: 251-331
(Номер группы)

Место прохождения практики: Московский Политех

Отчет принят с оценкой: _____

Дата принятия отчета « ____ » _____ 20 ____ г.

Руководитель практики: _____
(ФИО, ученая степень, ученое звание)

« ____ » _____ 20 ____ г.
(дата) (подпись) (И.О. Фамилия)

Москва, 2026

ОГЛАВЛЕНИЕ

Обновите поле (F9), чтобы собрать оглавление.

ВВЕДЕНИЕ

Проектная практика по направлению 09.03.02 состоит из базовой и вариативной части. Базовая часть одинакова для всех и посвящена инструментам: системе контроля версий, оформлению документации в Markdown и публикации материалов проекта. Вариативная часть выбирается индивидуально, и для настоящей работы выбран вариант «практическая реализация технологии».

Тема взята из каталога build-your-own-х, раздел Network Stack: самостоятельная реализация стека TCP/IP. Выбор объясняется тем, что сетевой стек — один из немногих механизмов, которым программист пользуется ежедневно и при этом почти никогда не видит изнутри. Вызов функции `send` скрывает за собой сегментацию, вычисление контрольных сумм, разрешение адресов и конечный автомат соединения. Реализация этих шагов вручную превращает абстракцию в набор понятных операций над байтами.

Цель работы — построить работающий стек, который принимает сырой кадр Ethernet и формирует корректный ответ, а корректность подтверждается не собственным выводом программы, а независимым сетевым анализатором.

Отчёт описывает постановку задачи, устройство полученного решения и способы проверки. Исходный код, документация и эталонные захваты трафика размещены в репозитории: <https://github.com/Gaymocoder/gcs-tcpip>.

1 Общая информация о проекте

1.1 Название проекта

`gcs-tcpip` — учебная реализация стека TCP/IP поверх Ethernet на языке C стандарта C11.

Проект оформлен как статическая библиотека `gcsll_tcpip` и два приложения поверх неё. Первое, `GCS.Replay`, прогоняет стек по заранее записанному файлу захвата. Второе, `GCS.Tapd`, подключает стек к настоящему сетевому интерфейсу операционной системы.

1.2 Цели и задачи проекта

Цель проекта: реализовать минимальный, но функционально замкнутый сетевой стек, обрабатывающий кадры Ethernet и доводящий обмен до установленного TCP-соединения с передачей данных и корректным закрытием.

Для достижения цели поставлены следующие задачи:

- изучить форматы заголовков Ethernet, ARP, IPv4, ICMPv4 и TCP по первоисточникам, то есть по документам RFC, а не по пересказам;
- реализовать разбор и сборку каждого заголовка, включая вычисление контрольных сумм по алгоритму RFC 1071;
- реализовать таблицу соответствий ARP и обработку широковещательных запросов;

- реализовать конечный автомат ТСР в объёме, достаточном для приёма соединения, обмена данными и закрытия;
- обеспечить возможность запуска и проверки без административных привилегий;
- подтвердить корректность сформированных пакетов внешним сетевым анализатором, а не собственным выводом программы.

2 Общая характеристика деятельности организации (заказчика проекта)

2.1 Наименование заказчика

Федеральное государственное автономное образовательное учреждение высшего образования «Московский политехнический университет», сокращённое наименование — Московский Политех.

2.2 Организационная структура

Университет подведомственен Министерству науки и высшего образования Российской Федерации. Внутренняя структура строится по факультетам, факультеты делятся на кафедры. Практика проходила на кафедре информатики и информационных технологий факультета информационных технологий, которая является выпускающей для направления подготовки 09.03.02 «Информационные системы и технологии».

2.3 Описание деятельности

Основной вид деятельности организации — образовательный: реализация программ высшего образования, а также научно-исследовательская работа. Применительно к проектной практике кафедра выступает заказчиком в том смысле, что формулирует задание, определяет требования к результату и принимает работу.

Практическим интересом кафедры в задачах подобного рода является развитие у обучающихся прикладных компетенций, которые трудно получить из лекционного материала: работы с двоичными форматами данных, чтения нормативных документов и построения воспроизводимой процедуры проверки собственного кода.

3 Описание задания по проектной практике

3.1 Базовая часть

Базовая часть задания одинакова для всех обучающихся и включает четыре группы работ: освоение системы контроля версий Git и ведение репозитория с осмысленной историей; оформление всех материалов проекта в формате Markdown; подготовку материалов о ходе проекта; взаимодействие с организацией-партнёром. Завершается базовая часть составлением настоящего отчёта по установленной структуре.

Трудоёмкость практики составляет 72 академических часа, из которых на базовую часть отводится от 32 до 40 часов.

3.2 Вариативная часть

Вариативная часть допускает три варианта: кафедральное индивидуальное задание, практическая реализация технологии и вклад в открытый проект. Для настоящей работы выбран второй вариант.

Условия варианта требуют выбрать технологию из каталога build-your-own-х, изучить способ её построения с нуля, воспроизвести практическую часть и составить техническое руководство, ориентированное на начинающих, с пошаговыми инструкциями, примерами кода и иллюстрациями. Отдельным пунктом требуется модификация проекта, выбираемая самостоятельно.

В разделе Network Stack каталога представлены два источника, оба на языке С: руководство Beej по сетевому программированию и цикл статей «Let's code a TCP/IP stack». За основу взят второй источник, поскольку он описывает построение стека, а не использование готового API операционной системы.

Модификацией по отношению к исходному источнику стала замена способа подачи трафика. В оригинале стек работает только с живым устройством TUN/TAP, что требует привилегии CAP_NET_ADMIN и делает проверку невозможной. В настоящей работе стек отделён от источника кадров, и добавлен режим прогона по файлу захвата,

который выполняется без привилегий и даёт одинаковый результат при каждом запуске.

4 Описание достигнутых результатов по проектной практике

4.1 Архитектура решения

Стек оформлен как статическая библиотека, не зависящая от способа получения кадров. Точкой входа является функция `gcs_stack_handle_frame`, которая принимает буфер с кадром и, если требуется ответ, заполняет выходной буфер и возвращает его длину. Такое разделение позволяет подключать к одному и тому же коду и живой интерфейс, и файл.

Источником кадров в первом режиме служит устройство TUN/TAP. Оно создаёт виртуальный сетевой интерфейс, с одной стороны которого находится обычный сетевой стек ядра, а с другой — файловый дескриптор пользовательской программы.



TAP отдаёт кадр Ethernet целиком, TUN — только пакет IP

Рисунок 1 — Прохождение кадров через устройство TUN/TAP

Различие режимов существенно: TUN передаёт пакеты сетевого уровня, тогда как TAP передаёт кадры Ethernet целиком. Поскольку в работе разбирается и канальный уровень,

используется режим TAP, а флаг IFF_NO_PI отключает добавление служебного префикса.

```
ifr.ifr_flags = IFF_TAP | IFF_NO_PI;
strncpy (ifr.ifr_name, requested_name, IFNAMSIZ - 1);

/* Требуется CAP_NET_ADMIN. Без неё ядро отвечает EPERM. */
if (ioctl (_return->fd, TUNSETIFF, &ifr) < 0)
{
    int saved = errno;
    close (_return->fd);
    _return->fd = -1;
    errno = saved;
    return false;
}
```

Листинг 1 — Создание интерфейса TAP (src/gcs_tap.c)

4.2 Канальный уровень

Заголовок Ethernet II занимает 14 байт и содержит адрес получателя, адрес отправителя и поле типа вложенного протокола.

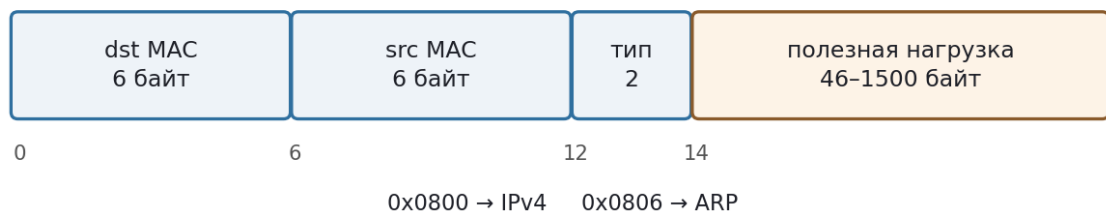


Рисунок 2 — Структура кадра Ethernet II

При разборе выявилась деталь, которая на схеме не видна. Минимальный кадр Ethernet составляет 60 байт без контрольной последовательности, поэтому короткие пакеты дополняются нулями до этой длины. Следовательно, длина кадра не совпадает с длиной полезных данных, и при разборе вложенного пакета IP опираться следует на поле общей длины из заголовка IP, а не на размер полученного буфера.

4.3 Разрешение адресов, ARP

Чтобы отправить кадр, необходим аппаратный адрес получателя, тогда как приложению известен только адрес сетевого уровня. Протокол ARP закрывает этот разрыв широковещательным запросом.

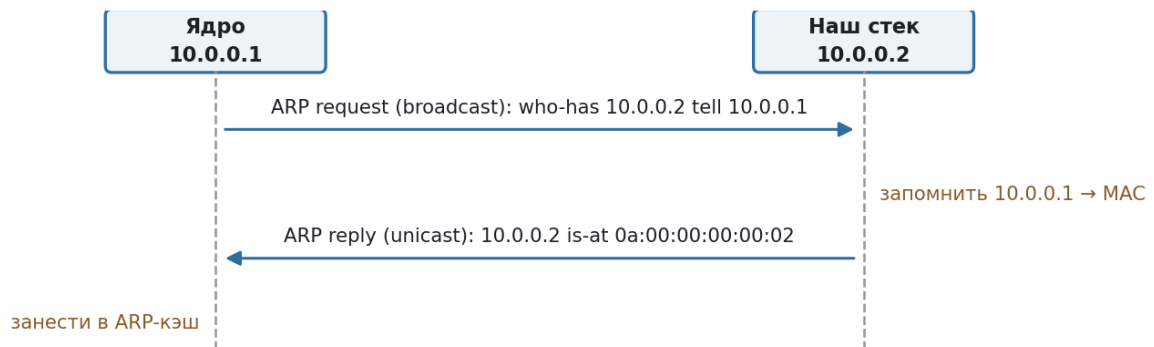


Рисунок 3 — Обмен ARP между ядром и реализованным стеком

Реализованный стек отвечает на запросы о собственном адресе, но сам разрешение адресов не инициирует, поскольку не устанавливает исходящих соединений. Поэтому таблица соответствий, которую RFC 826 предписывает вести, здесь не заводится: её нечем было бы читать. Соблюдается лишь требование отвечать одноадресно тому, кто задал вопрос.

4.4 Сетевой уровень и контрольная сумма

Заголовок IPv4 занимает не менее 20 байт. Поле длины заголовка хранит значение в 32-битных словах, поэтому при разборе его требуется умножать на четыре.

Контрольная сумма вычисляется по алгоритму RFC 1071: заголовок рассматривается как последовательность 16-битных

слов, они складываются, а переносы из старших разрядов прибавляются обратно к младшим, после чего результат инвертируется.

```
uint16_t gcs_checksum_finish (uint32_t sum)
{
    while ((sum >> 16) != 0)
    {
        sum = (sum & 0xFFFF) + (sum >> 16);
    }

    return (uint16_t) (~sum & 0xFFFF);
}
```

Листинг 2 — Свёртка переносов в контрольной сумме (src/gcs_checksum.c)

У алгоритма есть полезное свойство: если вычислить сумму по заголовку, в котором поле контрольной суммы уже заполнено, результат должен оказаться нулевым. Это позволяет свести проверку к одному сравнению и не обнулять поле перед пересчётом.

4.5 Протокол ICMP

Ответ на эхо-запрос отличается от самого запроса единственным байтом типа: значение 8 заменяется на 0. Идентификатор, номер последовательности и полезная нагрузка копируются без изменений, после чего контрольная сумма вычисляется заново. Это первый этап, на котором работа стека становится наблюдаемой извне: стандартная утилита `ping` начинает получать ответы.

4.6 Транспортный уровень, TCP

В отличие от предыдущих протоколов, TCP хранит состояние соединения. Реализованная часть конечного автомата

достаточна для приёма входящего соединения, обмена данными и корректного закрытия.

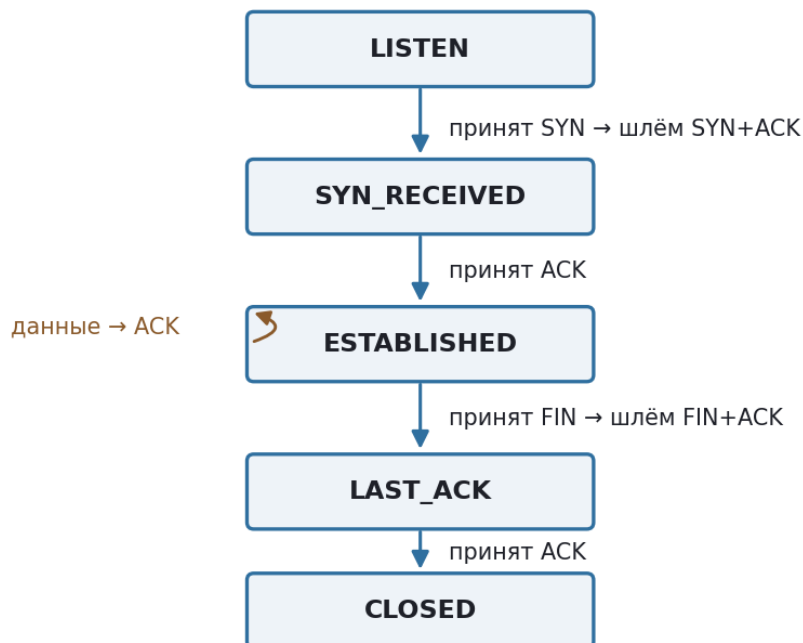


Рисунок 4 — Реализованная часть конечного автомата TCP

Номера последовательности считаются в байтах, при этом флаги SYN и FIN занимают по одной условной единице нумерации. Поэтому подтверждение на сегмент с флагом SYN равно номеру последовательности, увеличенному на единицу, а подтверждение на сегмент с данными увеличивается на их длину.

Наибольшую трудность представляет контрольная сумма ТСР. Она вычисляется не только по самому сегменту, но и по так называемому псевдозаголовку, который в передаваемом пакете отсутствует.

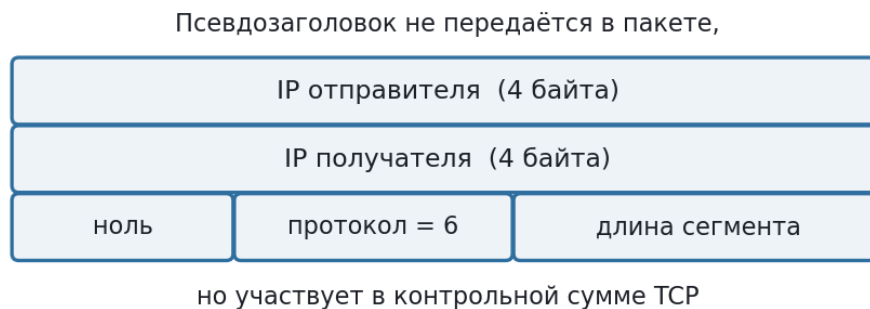


Рисунок 5 — Состав псевдозаголовка, участвующего в контрольной сумме TCP

Смысл конструкции в том, что заголовок IP защищает контрольной суммой только сам себя, но не полезную нагрузку. Включение адресов и номера протокола в сумму транспортного уровня позволяет обнаружить повреждение адресной информации.

```
uint32_t seed = gcs_pseudo_header_sum (src_ip, dst_ip, GCS_IP_PROTO_TCP,
                                       (uint16_t) total);
gcs_wr16be (out + 16, gcs_checksum_accum (out, total, seed));
```

Листинг 3 — Учёт псевдозаголовка при сборке сегмента (src/gcs_tcp.c)

4.7 Инфраструктура сборки

Система сборки опирается на шаблон CMakeAutoBuild, подключённый к проекту как подмодуль Git. Шаблон не копируется в проект, а остаётся внешней зависимостью, что позволяет получать его исправления командой обновления, не перенося правки вручную.

Устройство шаблона подчинено одному правилу: описание конфигураций существует в единственном экземпляре. Файл `presets.json` перечисляет наборы «платформа, компилятор, стандартная библиотека», а сценарий настройки порождает из него три artefacta: файл пресетов CMake, профили менеджера

зависимостей Conan и описание рабочего процесса непрерывной интеграции.

Замыкает конструкцию проверка в самой интеграции: перед сборкой сценарий настройки запускается заново, после чего выполняется сравнение полученного описания рабочего процесса с тем, что хранится в репозитории. Расхождение останавливает сборку. Таким образом единственность источника поддерживается механически, а не соглашением.

Прикладной эффект состоит в том, что каждое изменение проверяется семью сочетаниями компилятора и стандартной библиотеки, и каждое из них собирается с трактовкой предупреждений как ошибок. Это заменяет единственный доступный иначе критерий качества, состоящий в том, что программа запустилась. Практическая польза проявилась сразу: часть сочетаний отвергла сборку, которую локальный компилятор принимал, и различия пришлось устранять.

Шаблон рассчитан на C++, тогда как стек написан на C, поэтому пять его диагностик к языку неприменимы и дают только шум. Правка самого шаблона была бы стёрта при первом же обновлении, поэтому отличие вынесено в отдельный принадлежащий проекту файл, который снимает эти диагностики и включает санитайзеры в отладочной конфигурации.

Строгость набора оправдала себя дважды. Диагностика приведения, отбрасывающего квалификатор неизменяемости,

обнаружила дефект проектирования: функция разбора кадра принимала структуру с изменяемым указателем, из-за чего вызывающий код был вынужден отбрасывать этот квалификатор у входного буфера. Ошибка исправлена изменением сигнатуры, а не подавлением предупреждения. Второй случай оказался ошибкой в описании сборки: объявление подпроектов без явного указания языка включало компилятор C++ и вместе с ним набор диагностик, к языку C неприменимых.

```
Сборка через конвейер шаблона

$ GCST_WERROR=ON ./build.sh
|GCST| -- Reading presets from basic JSON: ".gcst/presets.json"
|GCST| -- -- Service elements found:
|GCST| -- -- -- .common-pre
|GCST| -- -- -- .common-post
|GCST| -- -- Presets found:
|GCST| -- -- -- unix-clang-libc++
|GCST| -- -- -- unix-clang-libstdc++
|GCST| -- -- -- win64-clang-libstdc++
|GCST| -- -- -- win64-clang-msvc
|GCST| -- -- -- unix-gcc-libstdc++
|GCST| -- -- -- win64-gcc-libstdc++
|GCST| -- -- -- win64-msvc-msvcstl
|GCST|
|GCST| -- Extracting CMake, conan and GitHub CI profiles from detected presets:
|GCST| -- -- unix-clang-libc++
|GCST| -- -- unix-clang-libstdc++
|GCST| -- -- win64-clang-libstdc++
|GCST| -- -- win64-clang-msvc
|GCST| -- -- unix-gcc-libstdc++
|GCST| -- -- win64-gcc-libstdc++
```

Рисунок 6 — Настройка и сборка проекта

4.8 Демонстрация работы

Проверка построена на двух независимых уровнях. Первый — прогон стека по эталонному захвату. Захват содержит восемь

кадров: широковещательный запрос ARP, два эхо-запроса ICMP, полную сессию TCP от установления соединения до закрытия, а также попытку подключения к закрытому порту.

Прогон стека: ARP, ICMP, начало TCP

```
$ ./bin/GCS.Replay --in data/sample.pcap --out data/replies.pcap
Интерфейс: ip=10.0.0.2 mac=0a:00:00:00:00:02, слушаем TCP-порт 8080
Вход: data/sample.pcap

[1] кадр 60 Б
  Ethernet aa:bb:cc:dd:ee:01 -> ff:ff:ff:ff:ff:ff тип=0x0806 (ARP)
  ARP request: who-has 10.0.0.2 tell 10.0.0.1 (aa:bb:cc:dd:ee:01)
  -> ARP reply: 10.0.0.2 is-at 0a:00:00:00:00:02

[2] кадр 60 Б
  Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
  IPv4 10.0.0.1 -> 10.0.0.2 протокол=1 (ICMP) ttl=64 len=36
  ICMP echo-request id=4660 seq=1 len=8
  -> ICMP echo-reply id=4660 seq=1

[3] кадр 60 Б
  Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
  IPv4 10.0.0.1 -> 10.0.0.2 протокол=1 (ICMP) ttl=64 len=36
  ICMP echo-request id=4660 seq=2 len=8
  -> ICMP echo-reply id=4660 seq=2

[4] кадр 60 Б
  Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
  IPv4 10.0.0.1 -> 10.0.0.2 протокол=6 (TCP) ttl=64 len=40
  TCP 40001 -> 8080 [S] seq=1000 ack=0 len=0 состояние=LISTEN
  состояние: LISTEN -> SYN_RECEIVED
  -> TCP [AS] seq=12648430 ack=1001 len=0

[5] кадр 60 Б
  Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
  IPv4 10.0.0.1 -> 10.0.0.2 протокол=6 (TCP) ttl=64 len=40
  TCP 40001 -> 8080 [A] seq=1001 ack=12648431 len=0 состояние=SYN_RECEIVED
```

Рисунок 7 — Обработка ARP, ICMP и установление соединения TCP

```
Прогон стека: данные, FIN, RST
```

```
(продолжение)
```

```
состояние: SYN_RECEIVED -> ESTABLISHED
```

```
[6] кадр 82 Б
```

```
Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
```

```
IPv4 10.0.0.1 -> 10.0.0.2 протокол=6 (TCP) ttl=64 len=68
```

```
TCP 40001 -> 8080 [AP] seq=1001 ack=12648431 len=28 состояние=ESTABLISHED
```

```
данные (28 Б), эхо обратно
```

```
-> TCP [AP] seq=12648431 ack=1029 len=28
```

```
[7] кадр 60 Б
```

```
Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
```

```
IPv4 10.0.0.1 -> 10.0.0.2 протокол=6 (TCP) ttl=64 len=40
```

```
TCP 40001 -> 8080 [AF] seq=1029 ack=12648431 len=0 состояние=ESTABLISHED
```

```
состояние: ESTABLISHED -> LAST_ACK (получен FIN)
```

```
-> TCP [AF] seq=12648459 ack=1030 len=0
```

```
[8] кадр 60 Б
```

```
Ethernet aa:bb:cc:dd:ee:01 -> 0a:00:00:00:00:02 тип=0x0800 (IPv4)
```

```
IPv4 10.0.0.1 -> 10.0.0.2 протокол=6 (TCP) ttl=64 len=40
```

```
TCP 40002 -> 9999 [S] seq=500 ack=0 len=0 состояние=LAST_ACK
```

```
TCP: порт закрыт, шлём RST
```

```
-> TCP [AR] seq=0 ack=501 len=0
```

```
Обработано кадров: 8, отправлено ответов: 7
```

```
Состояние TCP: LAST_ACK
```

```
Ответные кадры записаны в data/replies.pcap
```

Рисунок 8 — Передача данных, закрытие соединения и итоговая статистика

Второй уровень — независимая проверка. Ответные кадры записываются в файл захвата, который затем читается сетевым анализатором tcpdump. Анализатор вычисляет контрольные суммы самостоятельно и помечает их результатом проверки.

```
Проверка ответов анализатором tcpdump

$ tcpdump -r data/replies.pcap -n -vv
18:33:20.000001 ARP, Ethernet (len 6), IPv4 (len 4), Reply 10.0.0.2 is-at 0a:00:00:00:00:02, length 28
18:33:21.000001 IP (tos 0x0, ttl 64, id 1, offset 0, flags [DF], proto ICMP (1), length 36)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 4660, seq 1, length 16
18:33:22.000001 IP (tos 0x0, ttl 64, id 2, offset 0, flags [DF], proto ICMP (1), length 36)
    10.0.0.2 > 10.0.0.1: ICMP echo reply, id 4660, seq 2, length 16
18:33:23.000001 IP (tos 0x0, ttl 64, id 3, offset 0, flags [DF], proto TCP (6), length 40)
    10.0.0.2.8080 > 10.0.0.1.40001: Flags [S.], cksum 0xcb66 (correct), seq 12648430, ack 1001, win 4096, length 0
18:33:25.000001 IP (tos 0x0, ttl 64, id 4, offset 0, flags [DF], proto TCP (6), length 68)
    10.0.0.2.8080 > 10.0.0.1.40001: Flags [P.], cksum 0xefb6 (correct), seq 1:29, ack 29, win 4096, length 28: HTTP
18:33:26.000001 IP (tos 0x0, ttl 64, id 5, offset 0, flags [DF], proto TCP (6), length 40)
    10.0.0.2.8080 > 10.0.0.1.40001: Flags [F.], cksum 0xcb2d (correct), seq 29, ack 30, win 4096, length 0
18:33:27.000001 IP (tos 0x0, ttl 64, id 6, offset 0, flags [DF], proto TCP (6), length 40)
    10.0.0.2.9999 > 10.0.0.1.40002: Flags [R.], cksum 0xc687 (correct), seq 0, ack 501, win 4096, length 0
```

Рисунок 9 — Проверка сформированных стеком кадров анализатором *tcpdump*

Все сегменты помечены признаком `correct`, что подтверждает корректность вычисления контрольных сумм, включая учёт псевдозаголовка. Это подтверждение получено внешним инструментом и не зависит от собственного вывода программы.

4.9 Ограничения реализации

Реализованный стек является учебным, и часть механизмов сознательно оставлена за рамками работы. Не реализованы сборка фрагментированных пакетов IP, повторная передача по таймеру и связанное с ней управление интервалом ожидания, скользящее окно и программный интерфейс сокетов. Начальный номер последовательности задан константой, что упрощает воспроизводимость прогонов, но неприемлемо для реального применения.

Режим живого интерфейса требует привилегии `CAP_NET_ADMIN`. При её отсутствии программа сообщает об этом явно и подсказывает способы решения.

```
Запуск живого TAP без CAP_NET_ADMIN

$ ./bin/GCS.Tapd --dev gcstap0
gcs_tap_open(gcstap0): Operation not permitted
Нужна привилегия CAP_NET_ADMIN. Варианты:
    sudo setcap cap_net_admin+ep ./GCS.Tapd
    либо запуск от root
Без неё используйте GCS.Replay и заранее записанный pcap.
```

Рисунок 10 — Диагностика при отсутствии необходимой привилегии

ЗАКЛЮЧЕНИЕ

В ходе проектной практики реализован учебный стек TCP/IP поверх Ethernet, включающий разбор и сборку заголовков канального, сетевого и транспортного уровней, таблицу соответствий ARP, обработку эхо-запросов ICMP и конечный автомат TCP в объёме, достаточном для полного цикла соединения.

Поставленная цель достигнута. Корректность подтверждена двумя независимыми способами: воспроизводимым прогоном по эталонному захвату и проверкой сформированных кадров внешним сетевым анализатором, который вычисляет контрольные суммы самостоятельно.

Практическая ценность работы для заказчика состоит не в самом стеке, поскольку рабочие реализации существуют в каждой операционной системе, а в освоенных приёмах. К ним относятся чтение нормативных документов вместо пересказов, работа с двоичными форматами без вспомогательных

библиотек и построение процедуры проверки, не зависящей от собственного вывода программы.

Отдельным результатом стало наблюдение о ценности строгих настроек компилятора. Набор предупреждений обнаружил дефект проектирования интерфейса, который не проявлялся при выполнении, поскольку приводил не к неверному результату, а к утрате гарантии неизменности входных данных.

Направления дальнейшего развития очевидны из перечня ограничений: повторная передача по таймеру, скользящее окно и программный интерфейс сокетов.

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Postel J. RFC 791. Internet Protocol. — 1981. — URL: <https://www.rfc-editor.org/rfc/rfc791>
2. Postel J. RFC 792. Internet Control Message Protocol. — 1981. — URL: <https://www.rfc-editor.org/rfc/rfc792>
3. Postel J. RFC 793. Transmission Control Protocol. — 1981. — URL: <https://www.rfc-editor.org/rfc/rfc793>
4. Plummer D. RFC 826. An Ethernet Address Resolution Protocol. — 1982. — URL: <https://www.rfc-editor.org/rfc/rfc826>
5. Braden R., Borman D., Partridge C. RFC 1071. Computing the Internet Checksum. — 1988. — URL: <https://www.rfc-editor.org/rfc/rfc1071>

6. Saminiir. Let's code a TCP/IP stack. — URL:
<https://www.saminiir.com/lets-code-tcp-ip-stack-1-ethernet-arp/>
7. Codecrafters. Build your own X. — URL:
<https://github.com/codecrafters-io/build-your-own-x>
8. Universal TUN/TAP device driver. Документация ядра Linux. —
URL:
<https://www.kernel.org/doc/html/latest/networking/tuntap.html>
9. Руководство пользователя tcpdump. — URL:
<https://www.tcpdump.org/manpages/tcpdump.1.html>
10. Исходный код проекта. — URL:
<https://github.com/Gaymocoder/gcs-tcpip>

ПРИЛОЖЕНИЯ

Приложение А. Структура репозитория

.gcst/	presets.json и сценарии конвейера (владеет шаблон)
cmake/gcst/	утилиты и наборы предупреждений (владеет шаблон)
cmake/gcs/	проектный слой поверх шаблона
external/	CMakeAutoBuild как подмодуль
include/gcstcpip/	публичные заголовки
src/	реализация, модуль gcsll_tcpip -> gcsll::tcpip
app/	GCS.Replay (pcap) и GCS.Tapd (живой TAP)
tools/	генератор эталонного захвата
data/	эталонные захваты
docs/	техническое руководство и иллюстрации

Листинг 4 — Дерево каталогов проекта

Приложение Б. Конечный автомат TCP

```
case GCS_TCP_LISTEN:
{
    if ((tcp.flags & GCS_TCP_SYN) == 0)
    {
        return 0;
    }

    c->remote_ip    = ip->src;
    c->remote_port  = tcp.src_port;
    c->rcv_nxt      = tcp.seq + 1;
    c->snd_nxt      = c->iss + 1;
    c->state        = GCS_TCP_SYN_RECEIVED;

    return gcs_stack_emit_tcp (stack, eth, ip, &tcp,
                              c->iss, c->rcv_nxt,
                              GCS_TCP_SYN | GCS_TCP_ACK,
                              NULL, 0, out, out_cap);
}
```

Листинг 5 — Обработка входящего SYN (src/gcs_stack.c)

Приложение В. Команды сборки и запуска

```
git submodule update --init --recursive
GCST_WERROR=ON ./build.sh --preset unix-gcc-libstdc++

python3 tools/gcs_make_sample.py data/sample.pcap
./bin/GCS.Replay --in data/sample.pcap --out data/replies.pcap
tcpdump -r data/replies.pcap -n -vv
```

Листинг 6 — Воспроизведение результатов работы